

Classifying Music Genre using Machine Learning Methods

Colton Benson, Finn Rowles, Tarush Mayani

Abstract

This research applies two fundamental machine learning methods to classify music genres. Using the widely accepted GTZAN music dataset, we utilized feature engineering and neural network deep learning models to predict the genres of pop, hiphop, and rock. Our feature engineering approach resulted in a 77% accuracy score, surpassing the 62.7% of our best deep learning model. Computational limitations were the largest cause of poor performance for deep learning models, with the inability to train and test on the entire song due to runtime a major roadblock. Feature engineering proved to be a far better approach for music genre classification without a supercomputer, being far more computationally efficient while still providing an acceptable accuracy score.

Code Repository Link: https://github.com/CoBens14/ML_Music_Genre

Introduction

Music genre classification, a vital part of Music Information Retrieval (MIR) and an established space within audio classification, continues to be a growing focus in Machine Learning. Labeling genre has important implications for music recommendation systems, organization systems, audio libraries, copyright management, and beyond. As the volume of digitally available music continues to grow, manual labeling of music's genre has become impractical. This has motivated the development of data-driven, supervised learning approaches to this classification task, often seen on platforms such as Spotify, YouTube, or SoundCloud. MIR and machine-learning approaches to audio classification stretch far beyond the classification of music, applying similar signal-processing and learning techniques to tasks with research in volcanic tremor analysis (Zahra, 2023), automatic birdsong recognition and ecological impact analysis (Tao et al., 2025), emergency event detection in single-person households (Jinwoo et al., 2020) and much more. While the extraction of sound's features extends far beyond music, an abundance of literature clearly draws on foundational MIR methods.

Previous literature specifically focusing on genre classification primarily takes deep-learning approaches. Zheng and Li use “discrete wavelet transform (DWT), mel frequency cepstral coefficients (MFCC), and short-time fourier transform (STFT) characteristics” with a parallel convolutional neural network to achieve an Accuracy Score of 96.07 on the GTZAN dataset (Zhang and Li, 2025). Ahmed and colleagues take a similar approach, primarily extracting MFCC features, and achieving a classification accuracy of 92.7% with a modified convolutional neural network (Ahmed et al., 2024). The purpose of this report is to investigate the performance of machine learning methods using both raw time-domain information and extracted features. We aim to evaluate how both deep learning and traditional classification models perform on a labeled dataset of music recordings. We hypothesize that models trained on time-frequency domain expanded features will perform better than those trained on raw time-domain information. Subsequently, we theorize that ensemble classifiers will perform better than traditional linear classifiers, with deep-learning methods outperforming both of these strategies.

Methods

The dataset used for this study was the widely accepted GTZAN dataset, made publicly available on Kaggle. This dataset has remained a standard reference dataset for audio classification research over its decades-long existence. The GTZAN dataset consists of 1000 audio recordings, equally distributed between ten genres: blues, classical, country, disco, hip hop, jazz, metal, pop, reggae, and rock. Each recording is stored as a 30 second long .wav file. Audio recordings were collected from 2000-2001 from a medley of sources (such as personal recordings, CDs, or radio) to allow for varied recording conditions, creating a robust sample for feature extraction. The audio data, presented as individual files, consist of single-channel signals, each sampled at a rate of roughly 22,050 samples per second. We chose to focus on three genres: pop, hiphop, and rock. There were three primary reasons for this decision. Firstly, this regulated the computational power that would've been required for ten genres. These were also the genres present in the pretrained deep learning model that we used as a baseline. Finally, these three genres are among the most popular music genres worldwide.

Once these genres were selected, three repositories (pop, hiphop, rock) each populated with 100 .wav file samples were installed. Audio preprocessing and feature extraction were performed using the librosa Python library. All audio files were loaded at the default sampling rate and converted into univariate numerical time-series representations for further analysis. We experimented with a variety of features. To capture pitch/structural components, we utilized the Probabilistic YiN (*librosa.pyin*) algorithm (Mauch and Dixon, 2014) for robust fundamental frequency (F0) analysis. To capture rhythmic components, we created a tempo feature (*librosa.feature.tempo*). To create a “loudness profile”, we performed the root mean square energy transformation (*librosa.feature.rms*), which squares and averages instantaneous signal values. A similar approach was taken for spectral centroids, which relate to timbre and instrumentation. This transformation (*librosa.feature.spectral_centroid*) utilizes a particular short-time fourier transformation to extract where energy is concentrated in the frequency spectrum. Feature expansion was performed on each transformed output, for entire songs and equally distributed “frames”. A list of these features and further descriptions can be found in Appendix Element 1. A

visualization of an audio file object and each transformation's result can be found in Appendix Element 2. An example of a couple final feature-transformed observations are provided in Appendix Element 3. Once feature extraction was performed, the resulting dataset was partitioned into training and testing sets using an 80/20 split. Five-fold cross-validation was employed on the training data to select the model's hyperparameters. This approach allowed for a robust performance estimate and lessened the risk of overfitting. We performed three five-fold cross-validated grid searches for hyperparameter selection on different splits of the data: one with all features extracted from the entire file excluding pitch-related features, one with features extracted from three equal time partitions of the entire wav file, and one with all features extracted from the entire file including pitch-related features.

To test how deep learning can be used to predict music genre classification, multiple approaches were used. First, a deep learning model pretrained for music genre classification will be tested on all audio. Then, we trained two deep learning models for time series data, appropriate for the time series frequency data. A fine tuning approach using random weights was used since the models were not trained for music genre classification. As previously mentioned, both of these approaches were applied to the subset of genres of rock, pop, and hiphop due to computing and model limitations. Due to similar limitations, only a subset of the thirty second audio clips were used for training and testing. The section of song that was chosen varied as another parameter that was tested. The performance of the model was tested on a random created testing split after training had occurred. Some testing included a validation set in order to stop training early, but a validation set was not always used due to the added runtime.

Results

We first tested a deep learning model pretrained to classify music genres. The “distilhubert-finetune- music-genres-small” model was trained on 10 genres of music, which varied some from the genres in our data (Lewtun/Distilhubert-frozen-finetuned-music-genres-small). The three overlapping music genres are pop, rock, and hip-hop. Thus, these three genres were used to test the pretrained model without modifications. We used the highest probability of those three genres as the

model's predictions for testing. This model correctly predicted the genre of 144/300 songs that it was tested on, a 48% classification accuracy score (Figure 1). With three genres, random guessing would likely get 100/300 correct for a 33% accuracy score. This 48% score served as a baseline for our feature engineering and fine tuning of deep learning models approaches.

Deep learning had mixed results. The approach that led to the best results was with the Inception Time, a time series convolution neural network model from the TSAI library, and utilizing random starting weights. Using a learning rate of 0.002, 12 epochs, the first three seconds of each song, and a random training set of 180 songs and validation/testing sets of 60 observations each, we were able to achieve a model that got a 62.7% accuracy score on the testing set. However, the majority of models tested were unable to achieve better than random guessing on testing data, with only three models of the 20+ tested performing better than the baseline 48%. This is mainly due to the size of the audio files, which only allowed us to use about three seconds of audio, leading to high variability in model performance.

After our deep learning approach, we moved to a feature expansion approach. As previously discussed, through a grid search of hyperparameter combinations across XGBoost, Random Forest, SVM, Logistic Regressor, and K-Nearest Neighbors models, we found that the Random Forest ensemble performed best across each subset of different selected features. The best-performing model did so on the largest array of features, which included each feature decided in expansion. This cross-validated grid search obtained a Random Forest Classifier model with hyperparameters `max_features = 1`, `min_samples_leaf = 2`, `n_estimators = 200`, and `max_depth = None` achieving a cross-validated classification accuracy of 0.770833. Leaving a hold-out test set for robustness, we achieve a testing set accuracy of 0.771. These results can be seen in Figures 2-3. Additionally, for the two other feature-expanded data sets, we achieved testing accuracies of 0.758 and 0.771. These results can be seen in Figures 3 and 4. Each cross-validated grid search's best model obtained extremely similar hyperparameters and performances - high depth, small feature samples for trees, and small minimum sample leaves.

Discussion

Our feature engineering approach led to a 77% accuracy score and our best deep learning approach led to a 62.7% accuracy score. Feature engineering proved to be the more effective approach of the two we attempted. One reason for this is computing limitations, as deep learning models using only three seconds of audio would take 8-12 hours to train 50 epochs without a validation set, and could only complete around 10 epochs in that timeframe with a validation set. Through feature expansion, the Random Forest classifier demonstrated strong and stable performance across cross-validation and held-out testing, indicating reliable generalization for music classification.

The strong performance of the Random Forest model is likely due to its ability to help capture nonlinear relationships among spectral, rhythmic, and energy-based features, which likely differ systematically across pop, hip hop, and rock. Figures 2-7 provide confusion matrices representing both the cross-validated prediction accuracy and the hold-out set classification accuracies across feature mixes. Notably, the models consistently perform better on Rock music (with a mean class accuracy around 86%). This indicates that rock likely has more distinguishable spectral, rhythmic, and energy features. This aligns with what one might hear when listening to rock, with rock pieces often being fast-paced, with traditionally aggressive instrumentation and higher energy. Comparatively, while pop and hip-hop are distinct genres, there can be a lot of overlap in energy profile, sound, speed, and tone. We predict that performing this analysis on increasingly distinct genres (i.e. metal, classical, and country) would lead to a higher classification accuracy.

While both approaches performed better than the baseline deep learning model that we used as a comparison, there are still improvements that could be made. However, limitations around the length of audio files led to heavy computational burdens. These limitations proved too challenging to overcome to achieve the high accuracies that some research papers have gotten on this data, but this analysis clearly indicated that both feature engineering and deep learning could learn how to classify music genres and we would expect that these approaches combined with more powerful computing hardware would be able to produce high level performance.

References

Lewtun/Distilhubert-frozen-finetuned-music-genres-small.

<https://huggingface.co/lewtun/distilhubert-frozen-finetuned-music-genres-small/discussions>

"librosa: Audio and Music Signal Analysis in Python." Version 0.11.0 librosa.org/doc/latest/index.html

Olteanu, Andrada. "GTZAN Dataset - Music Genre Classification." *Kaggle*, Kaggle, Inc.,
www.kaggle.com/datasets/andradaolteanu/gtzan-dataset-music-genre-classification/data

Ahmed, Mumtahina, et al. "Musical genre classification using advanced audio analysis and Deep Learning Techniques." *IEEE Open Journal of the Computer Society*, vol. 5, 2024, pp. 457–467, <https://doi.org/10.1109/ojcs.2024.3431229>.

Hu, Tao, et al. "Avian vocalizations in Huangmaohai Sea-crossing channel: Automatic birdsong recognition and ecological impact analysis based on Deep Learning." *Biological Conservation*, vol. 305, May 2025, p. 111101, <https://doi.org/10.1016/j.biocon.2025.111101>.

Kim, Jinwoo, et al. "Occupant Behavior Monitoring and Emergency Event Detection in Single-Person Households Using Deep Learning-Based Sound Recognition." *Building & Environment*, vol. 181, Aug. 2020, p. N.PAG. *EBSCOhost*, <https://doi.org/10.1016/j.buildenv.2020.107092>.

Mauch, Matthias, and Simon Dixon. "Pyin: A fundamental frequency estimator using probabilistic threshold distributions." *2014 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, May 2014, pp. 659–663, <https://doi.org/10.1109/icassp.2014.6853678>.

Zali, Zahra. "Automatic transient signal detection and volcanic tremor extraction using Music Information Retrieval Strategies." *RILM Abstracts of Music Literature*, Jan. 2023, <https://research.ebsco.com/c/2q624j/search/details/jbxtlf65qn?db=ahl%2Ch9h%2Ch9i%2Ch9j%2Ch9k%2Ch9m%2Canl%2Casu%2Ch6a%2Ch6b%2Creh%2Cnlebk%2Ce001mww%2Cecn%2Cehh%2Chev%2Cf3h%2C8gh%2Chia%2Chus%2Clxh%2Cmsn%2Ckah%2Cmzh%2Cddu%2Cram%2Ctrh&limiters=None&q=music%20information%20retrieval&searchMode=all>

Zhang, Yuxin, and Teng Li. "Music genre classification with parallel convolutional neural networks and capuchin search algorithm." *Scientific reports* vol. 15,1 9580. 20 Mar. 2025, https://www.researchgate.net/publication/390039787_Music_genre_classification_with_parallel_convolutional_neural_networks_and_capuchin_search_algorithm

Appendix

Element 1

Feature Descriptions:

F0_nan_ratio: The Probabilistic YiN algorithm takes fundamental frequencies as input. This indicates the proportion of analysis frames for which no fundamental frequency could be estimated, indicating the presence of unpitched, noisy, or percussive content. For PYiN, we searched for frequencies between a C2 and a C6, which are generally considered to be about an octave below and an octave above the scope of a normal singer's range (C3-C5).

F0_num_gaps: The number of gaps in the frequencies, where PYiN pitch estimation failed. Reflects pitch discontinuity.

o1, o2, o3, o4: Ultimately dropped as features due to a lack of predictive power, representing the harmonic ratios at each quarter slice of the song, corresponding to "harmonicity".

Tempo: Estimated tempo of the audio, measured in beats per minute (BPM)

Rolling features:

Mean_freq: The mean estimated fundamental frequency across all frames, representing average pitch height. NAs are not considered.

Std_freq: The standard deviation of estimated fundamental frequency across all frames. NAs are not considered.

Max_freq: The maximum estimated fundamental frequency from all frames, representing average pitch height. NAs are not considered.

Rms_mean: Mean RMS energy across the full audio segment.

Rms_std: Standard deviation of the RMS energy across the full audio segment.

Rms_min: Minimum RMS energy across the full audio segment.

Rms_max: Maximum RMS energy across the full audio segment.

Centroid_mean: Mean spectral centroid across the audio segment. Reflects "instrumentation" or "timbral" mean.

Centroid_std: Standard deviation of the spectral centroid, reflecting timbral variability.

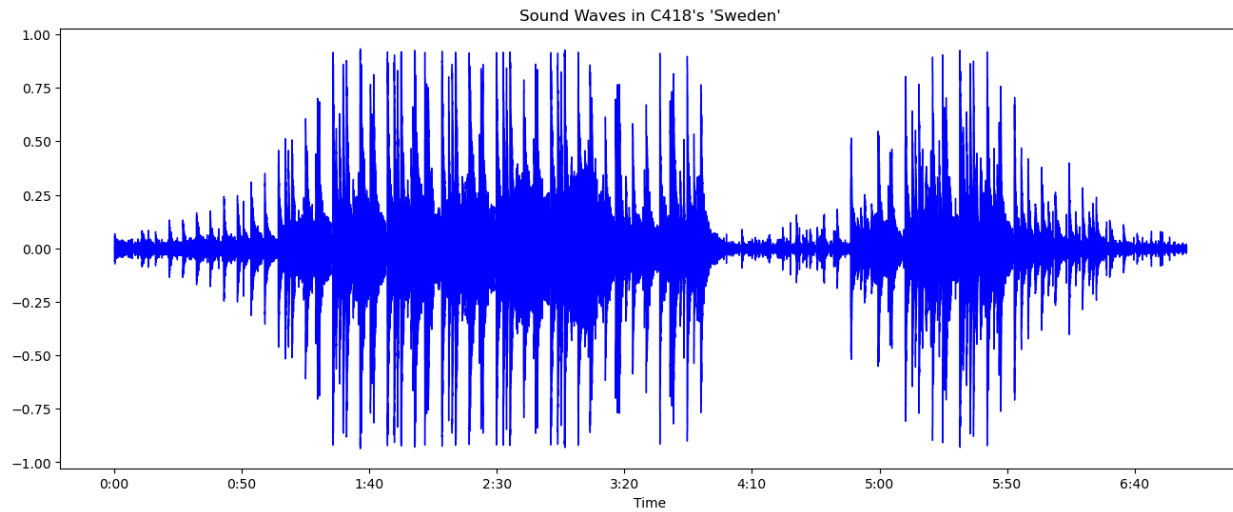
Centroid_min: Minimum of the spectral centroid.

Centroid_max: Maximum of the spectral centroid.

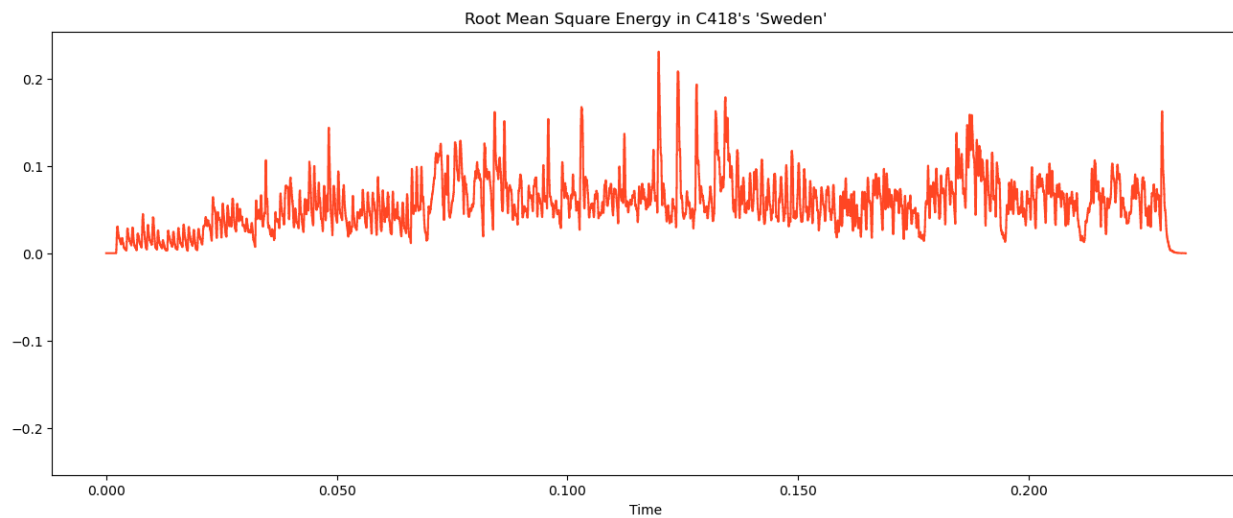
Feature_nth_slice: For each rolling feature, calculate the value of the n (1-nth) fold of the song. For example: Centroid_max_3rd_slice

Element 2

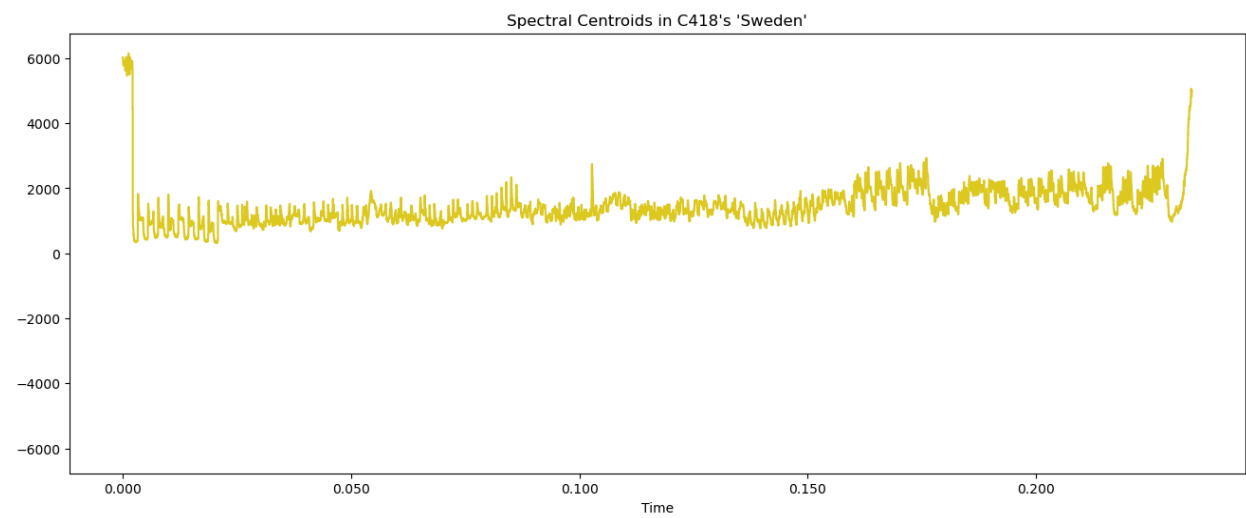
A raw audio file, loaded through Librosa and visualized over time



The root mean square energy transformation on the same audio file: notice how the x (time) axis is different, but the energy profile roughly aligns with the “sound” or “noisiness” going on



The spectral centroid, obtained from a short-time fourier transformation. Since this piece is mostly piano, there is not much erratic variation in spectral centroid over time.



Element 3

Feature vector representation for observation 0 (a pop song) for all features: INCLUDING rolling window stats for song thirds (ten second snippets)

Feature	Observed Value
f0_nan_ratio	0.458623
f0_num_gaps	593
mean_freq	101.794192
std_freq	91.230777
max_freq	460.809414
o1	0.957815
o2	1.915631
o3	2.736616
o4	3.694431
tempo	135.999178
rms_mean	0.185783
rms_std	0.086690
rms_min	0.023249
rms_max	0.492508
centroid_mean	3422.071864
centroid_std	1039.287259
centroid_min	509.940032
centroid_max	7532.942458
rms_mean_first_slice	0.159071
rms_std_first_slice	0.075696
rms_min_first_slice	0.027823

rms_max_first_slice	0.329526
centroid_mean_first_slice	3716.140957
centroid_std_first_slice	1127.404283
centroid_min_first_slice	1093.175046
centroid_max_first_slice	7532.942458
rms_mean_second_slice	0.170023
rms_std_second_slice	0.084919
rms_min_second_slice	0.023249
rms_max_second_slice	0.379536
centroid_mean_second_slice	3317.164425
centroid_std_second_slice	1144.103364
centroid_min_second_slice	509.940032
centroid_max_second_slice	6672.065467
rms_mean_third_slice	0.228256
rms_std_third_slice	0.082696
rms_min_third_slice	0.072351
rms_max_third_slice	0.492508
centroid_mean_third_slice	3232.910210
centroid_std_third_slice	726.000527
centroid_min_third_slice	1297.260328
centroid_max_third_slice	5723.421717

Feature vector representation for observation 299 (a rock song) for entire file features

Feature	Observed Value
f0_nan_ratio	0.602475
f0_num_gaps	779.000000
mean_freq	251.816350
std_freq	81.893525
max_freq	359.461400
tempo	107.666016
rms_mean	0.153637
rms_std	0.122112
rms_min	0.011199
rms_max	0.503713
centroid_mean	1963.064382
centroid_std	821.914877
centroid_min	645.558214
centroid_max	5429.444218

Figure 1
Classification Matrix - Distill Hubert Baseline Model

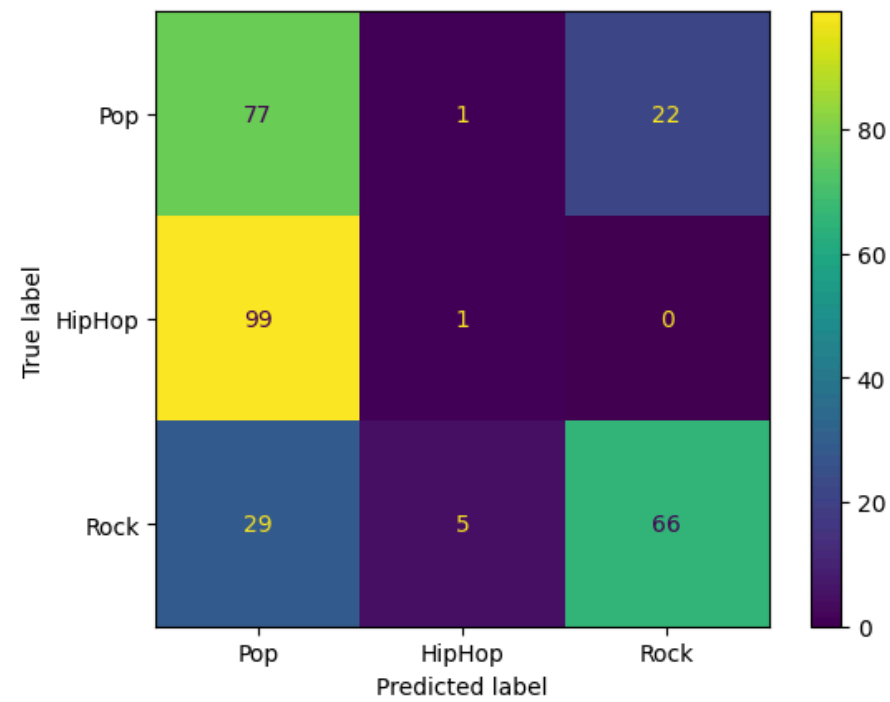


Figure 2
Classification Matrix - Cross Validated Training - All Features

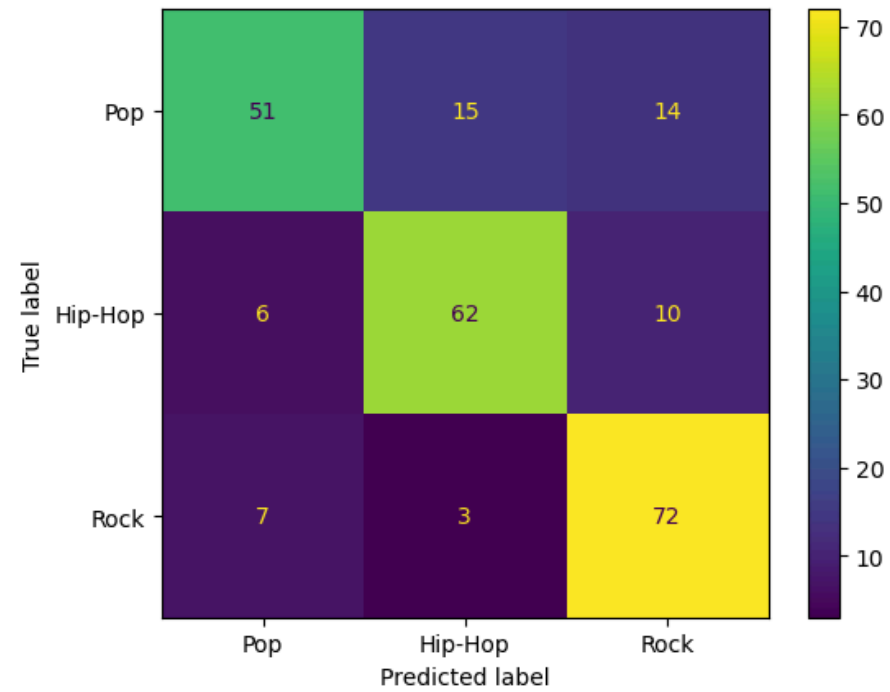


Figure 3
Classification Matrix - Testing - All Features

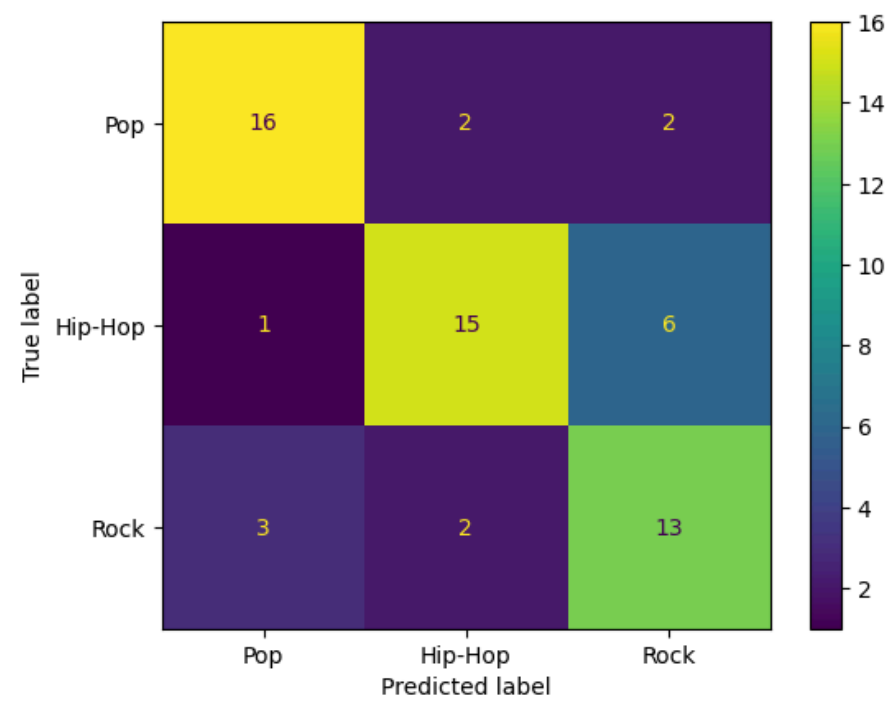


Figure 4
Classification Matrix - Cross Validated Training - Without Overtone Extractions

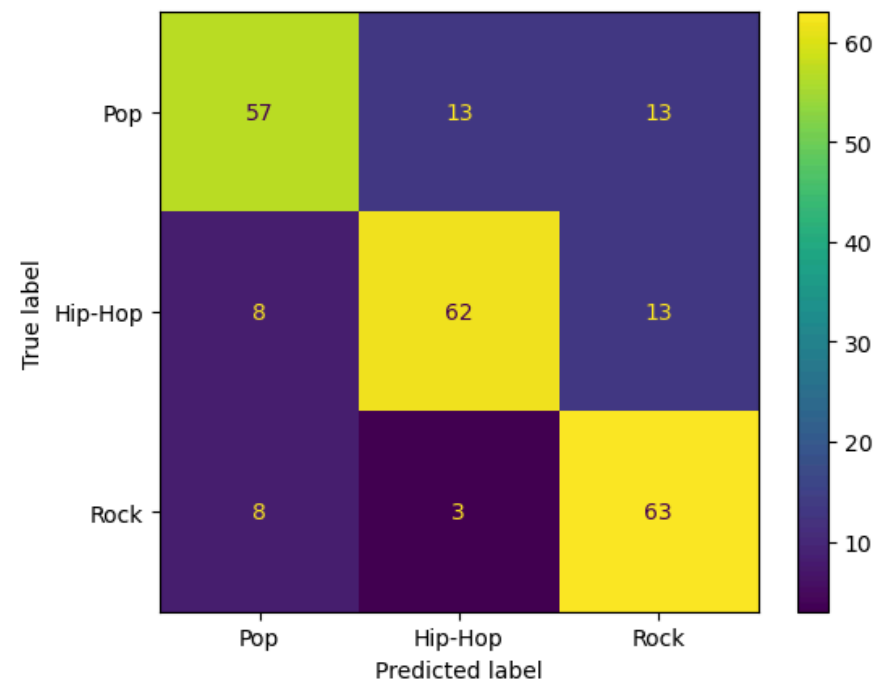


Figure 5
Classification Matrix - Testing - Without Overtone Extractions

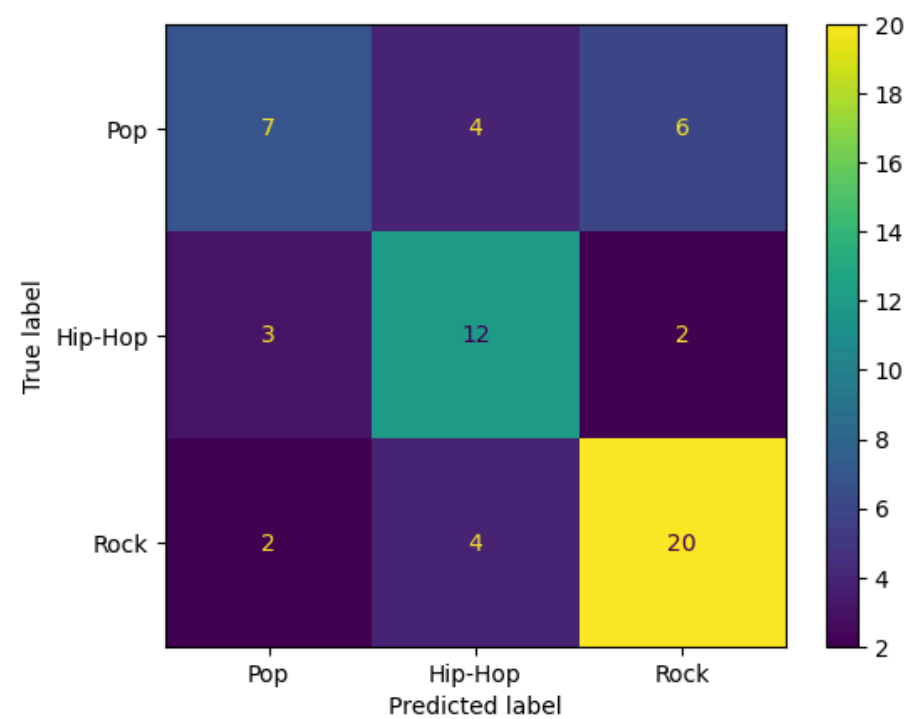


Figure 6
Classification Matrix - Cross Validated Training - Only Full Song Rolling Stats

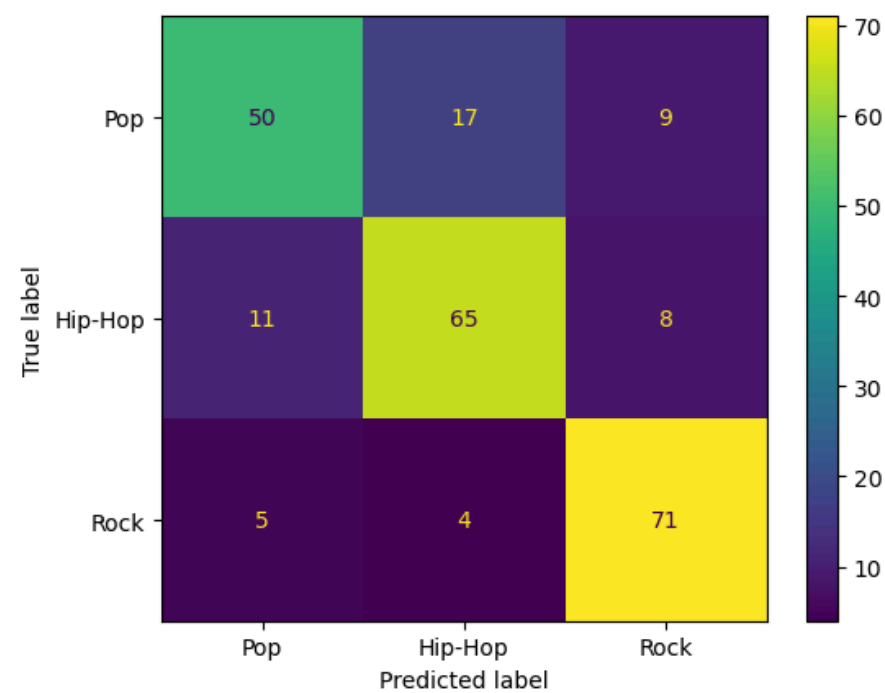


Figure 7
Classification Matrix - Testing - Only Full Song Rolling Stats

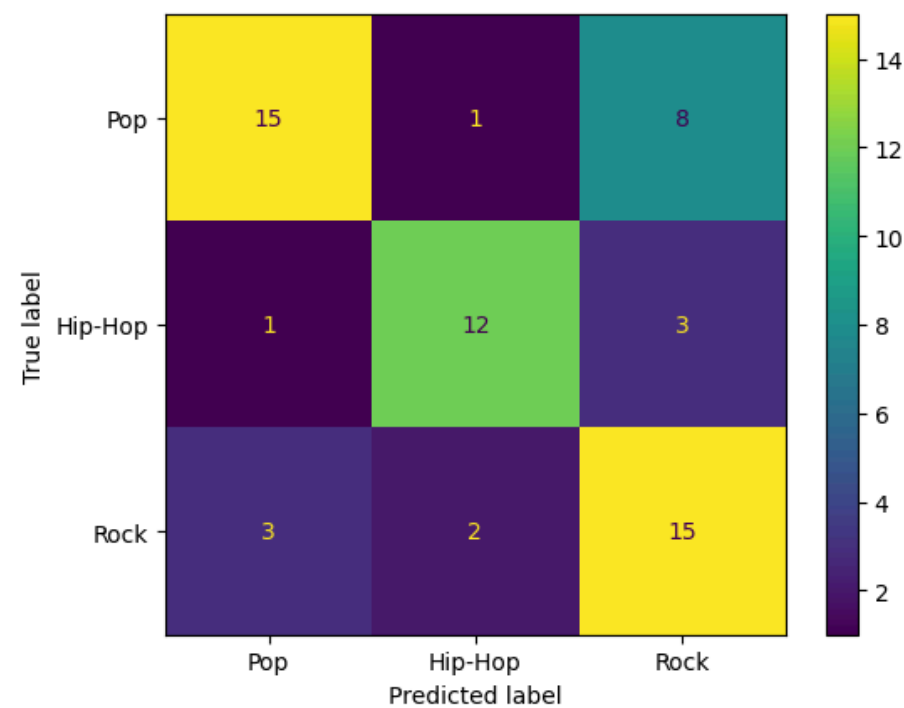


Figure 8
Training and Validation Cost Function Graph Example - 83% Training Accuracy - 45% Testing Accuracy

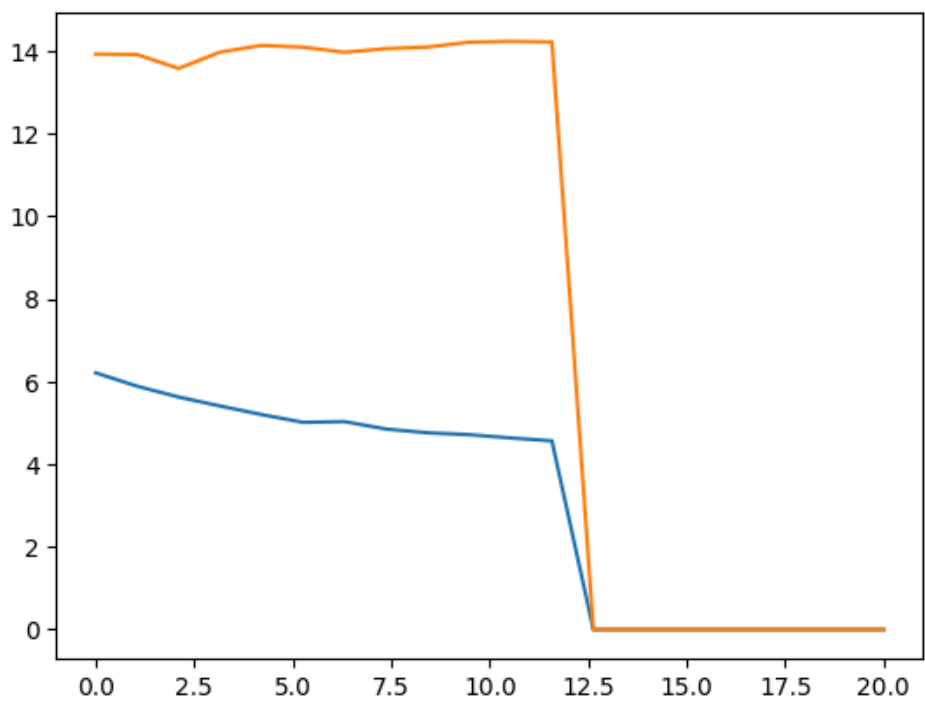


Figure 9

